

Face recognition

Oscar Brault
IA et Deep Learning
ESIEE - 2025
France
oscar.brault@
edu.esiee.fr

Victor Chen
IA et Deep Learning
ESIEE - 2025
France
victor.chen@e
du.esiee.fr

Abstract—This project aims to use the Python language as well as tools such as OpenCV and dlib to develop a facial recognition tool. The objective of this project is to detect, align, encode faces from images and videos in order to recognize them with high accuracy. The challenges were to manage the conditions of our model, orientation of faces while having the best performances including the speed of the model and its accuracy. For this we used detection models such as HOG and CNN, facial landmark predictors, and a ResNet-based face encoder. We trained different classifier on encoded faces to perform on the identity prediction such as SVM, Knn, logic regression and neural network. We aim to build a reliable and efficient recognition pipeline adaptable to different input formats.

Index Terms—Face recognition, Deep Learning, image and video processing, Face landmarks and classifier.

I. INTRODUCTION

Whether for public security, smartphone authentication, or many other applications, facial recognition has become a key element for many applications over the years. Although it is becoming more widespread, it still faces many obstacles that compromise its accuracy and reliability. These include pose changes and the difficulty of obtaining features to process from unprocessed facial photos.

Facial orientation is a major challenge in facial recognition. Indeed, a model trained on frontal photos may perceive a face tilted differently. To achieve this, we use facial landmark identification to normalize poses and allow us to align facial features such as lips, eyes, and others across different photos.

Effectively modeling a face for classification without using large convolutional networks, which are difficult to leverage, is another challenge. We encode each face into a compact 128-dimensional vector that contains facial measurements extracted using a deep learning model rather than directly classifying them. Even with limited computing resources, our encoding efficiently and robustly captures key facial features, enabling faster and more accurate detection.

II. RELATED WORKS

We can find in the following articles two works of facial recognition, one using traditional methods such as PCA and Viola-Jones and another using deep neural networks, we will dedicate a brief description of each work. [1]

Face Recognition Using PCA and the Viola-Jones Algorithm: This face recognition algorithm uses a

combination of PCA for feature extraction and the Viola-Jones algorithm for face detection. [1]

It uses the Carnegie Mellon University database, which includes images with variations in expression and pose. The system accuracy improves by integrating PCA with the Viola-Jones algorithm, but it still experiences challenges with variations in pose and expression and fails to recognize eyes. A possible improvement would be the implementation of facial alignment techniques to reduce these effects. [1]

It is a facial recognition system based on deep neural networks that learns a compact representation of facial images. It uses the Labeled Faces in the Wild (LFW) database and achieves an accuracy of $99.63\% \pm 0.09$. However, although the results are outstanding, the model requires a large amount of data and computational resources. [2]

III. METHOD

In this work, a facial recognition system is developed by performing a facial recognition model trained from a custom database, we use "python" as the programming language and the packages "Dlib", "OpenCV", "Keras", "Numpy", "IPython", "scikit-learn" and "os". The different stages in the realisation of the project will be described below.

A. Face detection

Face detection is the beginning of everything that facial image analysis is about, it is the first point to consider when creating human-machine interaction systems based on vision. [3]

The data used in this project consisted of face images of varying dimensions, organized into separate directories for everyone. In total, the dataset included approximately 70 images per individual.

Face detection was a critical preprocessing step, as it allowed us to isolate and extract only the faces from larger images, ensuring that the model only considered the necessary parts.

We used HOG- and CNN-based detectors from the "dlib" library to localize faces and extracted the faces before resizing them to 128x128 pixels and with three color channels. After extraction, all images were normalized to a [0, 1] scale, and labels were encoded into numbers using "LabelEncoder," followed by one-hot encoding for compatibility with the softmax output layer. The dataset was then split into 70% training and 30% testing. (For classification, we implemented a lightweight convolutional

neural network composed of two convolutional layers with increasing filter sizes, followed by max pooling, flattening, a dense hidden layer with 128 units, and a softmax output.).

This architecture was chosen for its simplicity and enough ability to learn distinctive features on a small dataset.

The model achieved an accuracy of 74% on the test set. Experimenting with changes, such as adding layers, increasing the number of hidden units, or modifying the learning rate, produced slight improvements, but training times became longer or overfitting without additional regularization.

For pose estimation, we worked with face images previously extracted using face detection. To normalize facial orientation and improve the dataset, we implemented a face alignment technique based on landmark detection using dlib's 68-point shape predictors. These models allow us to locate facial features such as eyes, nose, and mouth, which we subsequently used to align all faces to a specific pose.

We used the coordinates around the eyes, nose, eyebrows, lips, and jaw as landmarks, warping the images according to a predefined "MINMAX_TEMPLATE" template. The distribution between training and testing remained the same as before, allowing for a comparison between the datasets.

After pose correction, we retrained the previous convolutional neural network and noticed an improvement in classification accuracy, suggesting that the alignment reduced the variance due to head tilt and rotation.

The improvement is due to the network focusing on the consistency of feature locations, making the learning process more efficient and reducing overfitting to pose variations.



Fig. 1. Faces before and after prediction.

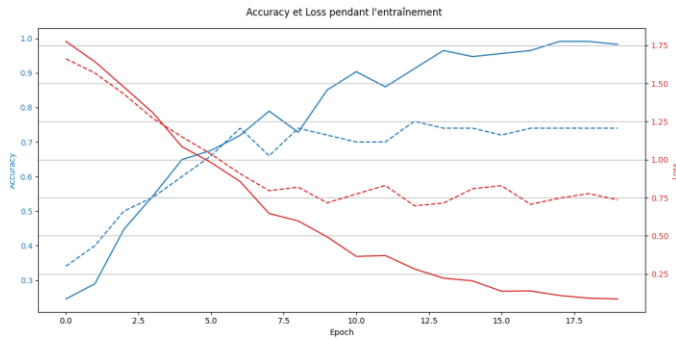


Fig. 2. Accuracy and loss on train set(-) and test set(--).

B. Pose estimation

Pose estimation is the process of determining the orientation and position of a face in an image by identifying key landmarks such as the eyes, nose, mouth, and jaw. In the context of face recognition, head pose estimation is necessary to normalize facial images and improve the accuracy of tasks such as expression recognition or analysis. Facial pose estimation can be performed using regression-based models that predict the location of landmarks, such as the regression tree ensemble method described in their work. [4]

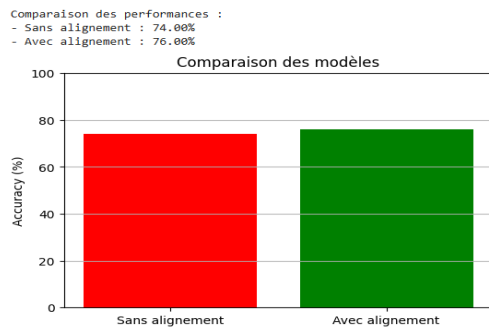


Fig. 4. Comparison of performances.

C. Face encoding

Write as the first paragraph what is Encoding - if you can be specific try to explain in the context of face encoding- (try to cite 1 work that you got this information from)

Face encoding is the process of transforming a human face from an image into a digital representation represented by a vector that captures the facial features. These measures encode abstract facial features to enable fast and efficient comparison between faces [2]. The network is trained using a “triplet loss” mechanism: it collects encodings from the same person and separates those from different individuals, making the resulting facial representations highly discriminatory.

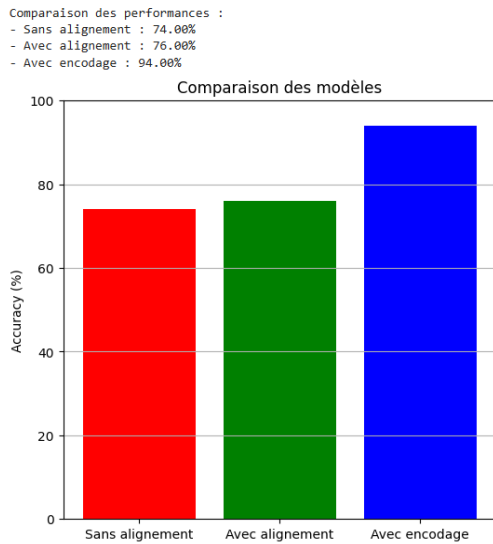


Fig. 5. Comparison of performances.

The main advantage of face encoding is its efficiency and scalability. Instead of training a large classification model that would have to be retrained each time new people were added, we simply compare vectors representing faces. Once a face is encoded into a 128-dimensional vector, recognizing a person is as simple as calculating distances between the vectors. This significantly reduces computational costs and storage requirements, while allowing the system to handle millions or even billions of faces without requiring retraining.

During face encoding, the data used are face images generally well-cropped, front-facing pictures of individuals. These images may come from various sources such as user photo uploads, surveillance footage, or datasets like Labeled Faces in the Wild (LFW). Ideally, the faces are already detected and aligned so that they are centered and normalized before being fed into the encoding network. Good quality and consistent preprocessing are critical because variations in lighting, angle, or occlusions can negatively affect the encoding quality.

After face encoding, each face image is transformed into a fixed-length 128-dimensional vector. This vector no longer resembles the original image but contains enough information to distinguish between different people. Importantly, similar faces will have vectors that are close to each other in the 128-dimensional space, while different faces will have vectors that are farther apart. This numerical transformation allows simple mathematical operations like computing the Euclidean distance to tell how similar two faces are.

In this step, we are no longer training a model to recognize faces directly from pixel images. Instead, if further classification is needed (such as identifying a face from a known database), we can train a much simpler classifier like k-nearest neighbors (k-NN), Support Vector Machines (SVM), or even a logistic regression model on top of the 128-dimensional face encodings. Because the encodings are already highly informative, these lightweight models perform very well with little data and training time.

With this method the results improve significantly when using face encodings compared to trying to classify raw face images directly. The main improvements are that we have a higher accuracy (because the CNN extracts more discriminative features than raw pixels or manually chosen measurements).

A faster training and inference (because encodings are compact and require less computation).

And the generalization is better (because the system can recognize unseen faces with high reliability).

D. Face recognition

Face recognition is the task of identifying or verifying a person based on their facial features. Unlike face detection, which simply finds the location of faces in an image, face recognition assigns an identity to a detected face by comparing it against a database of known faces.[5]

Face detection and face recognition are related but distinct processes. Face detection is about locating faces in an image, often by drawing bounding boxes around them. Face recognition happens after detection. It tries to determine who is it by comparing the detected face against known identities.

For the face recognition task, after obtaining the face encodings (128-dimensional feature vectors), I chose the classifiers k-Nearest Neighbors (k-NN), Support Vector Machine (SVM), Logistic Regression and Neural Network models. These were chosen because k-NN works extremely well when distances between feature vectors are meaningful. SVM is highly effective in high-dimensional spaces and is particularly good at classification tasks where classes are separable with clear margins. Logistic Regression is simple and fast, providing a good baseline to evaluate more complex models and Neural Network models can learn complex, non-linear relationships between features and are capable of generalizing well if trained properly, especially on more complex datasets.

```
Logistic Regression:
Accuracy: 92.00%
Training Time: 0.03s
Prediction Time: 0.00s

SVM:
Accuracy: 100.00%
Training Time: 0.01s
Prediction Time: 0.00s

kNN:
Accuracy: 100.00%
Training Time: 0.00s
Prediction Time: 0.05s

Neural Network:
Accuracy: 100.00%
Training Time: 0.40s
Prediction Time: 0.00s
```

Fig. 6. Comparison of models.

k-NN performed very well but can become computationally expensive at prediction time. SVM offered the highest overall

accuracy, combining speed and robustness. Logistic Regression was fast to train but less accurate due to its simplicity. Neural Network achieved nearly the same performance as SVM, showing strong generalization, especially when the training set was slightly more complex.

The best performance was with the SVM model. It gets an accuracy of 100% and the best speed compare to each other's. This can be due by the fact that it captured non-linear relationships between different identities, making the model robust to small variations.

E. Personal dataset

After all of this we created our own dataset by first taking videos of some people. With a python program on VSCode we extract around 100 images from each video (9 persons) and created a folder named by the name of the person to contain its. To finish the dataset, we convert it into a .rar format and import it to kaggle.

The SVM model is working on our personal set and reach an accuracy of 100%. The difference is in time which increased, but it's normal if we take in mind that there are more pictures and person than in the previous set. This can be bad if we want to create a large dataset which is containing 1000+ persons but in our case it's not

Our dataset has a better quality in the images than the previous and can be modify at any moment. For this we just need to add a new folder for the new person by updating (on Kaggle for example).

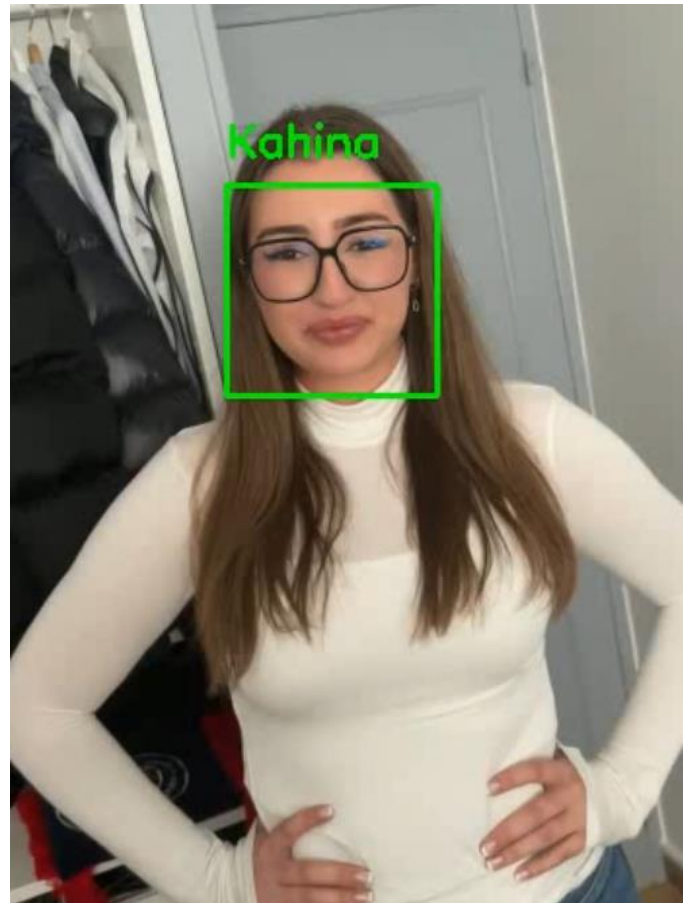


Fig. 7. Test with the SVM model.

F. Extra - Bias analysis

Bias in the context of machine learning refers to the systematic error introduced by an algorithm's assumptions, which lead it to produce results that are not representative of the real-world distribution of data. In face recognition, bias typically manifests when the model performs better for certain groups of people and worse for others. This can happen due to skewed training data or unaccounted-for features.

Bias is problematic because it leads to unfair or discriminatory results. In face recognition, for example, if a model is biased toward a specific ethnicity, it could wrongly classify people from other ethnicities, leading to a higher rate of false positives or false negatives. In critical areas like security, biased models can cause errors that disproportionately affect certain groups, creating ethical and legal concerns. Additionally, biased models do not generalize well, leading to poor performance when applied in diverse real-world scenarios.

Yes, diversifying input examples is crucial in mitigating bias. When collecting or using a dataset for face recognition, it is important to ensure that the data comes from a wide variety of sources, representing multiple ethnicities, genders, ages, and other demographics. If the dataset contains only faces from a single person, ethnicity, or group, the model will likely perform well only for that group, which would cause biased predictions for others. In my case, I ensured that the dataset includes diverse ethnicities and ages to better evaluate whether the model performed equally across groups.

To verify the model's fairness, I could compare its performance across different subgroups like ethnic or other, to check if there are disparities in accuracy, precision, recall, or F1-score based on ethnicity. This would reveal if the model has a bias toward a group.

When analyzing face detection data, it's essential to compute the following statistics to evaluate fairness and performance like the Accuracy which is the overall percentage of correct predictions, the false Positive Rate (FPR) and False Negative Rate (FNR) which indicate how often the model misclassifies faces, which can be examined by group, the Precision, Recall, and F1-Score: These provide deeper insights into the performance per. If there is a significant difference in the scores between groups, this could suggest bias.

In our own dataset, we ensured a balanced representation of different ethnicities and ages, which allowed me to observe whether the model exhibits any significant performance differences. This is crucial to avoid situations where the model might favor one demographic over others, especially when applied to real-world scenarios with a diverse population. For example, I checked if faces from underrepresented groups showed an increased rate of misclassification. If we had seen any such patterns, I would have considered strategies to balance the dataset further to improve the fairness of the model.

CONCLUSION

In this work, we explored the process of face recognition, from face detection and encoding to classification using various machine learning models. We began by extracting face encodings using pre-trained deep learning networks, which allowed me to represent each face as a 128-dimensional vector. We then employed several classifiers, including k-NN, SVM, Logistic Regression, and Neural Networks, to perform face recognition. Through experimentation, we learned that while simpler models like k-NN and Logistic Regression provided decent performance, SVM and Neural Networks achieved the best results, with SVM being the most accurate.

Furthermore, we also addressed the important issue of bias in machine learning, emphasizing how the diversity of training data impacts model fairness. We created a balanced dataset to ensure the model performed equally well across different demographics, and we learned that bias in facial recognition systems is a significant challenge. By analyzing model performance on diverse groups, we gained insights into how models can unintentionally favor certain groups, prompting further improvements to address fairness. Ultimately, this project not only deepened my understanding of face recognition systems but also highlighted the importance of ethical considerations in machine learning applications.

REFERENCES

- [1] J. Sebastian, "Reconocimiento facial mediante el uso de PCA y algoritmo de reconocimiento Viola-Jones Objetivos Objetivo principal," *Academia.edu*, Sep. 09, 2018.
- [2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2015, Doi: <https://doi.org/10.1109/cvpr.2015.7298682>
- [3] S. Zafeiriou, C. Zhang, and Z. Zhang, "A survey on face detection in the wild: Past, present and future," *Computer Vision and Image Understanding*, vol. 138, pp. 1–24, Sep. 2015, Doi: <https://doi.org/10.1016/j.cviu.2015.03.015>
- [4] V. Kazemi and J. Sullivan, "One millisecond face alignment with an ensemble of regression trees," 2014 IEEE Conference on Computer Vision and Pattern Recognition, 2014, Doi: <https://doi.org/10.1109/cvpr.2014.241>
- [5] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep Face Recognition," Doi: <https://doi.org/10.1109/CVPR.2015.7298682>